

HOMework 2

Gaurav Batra gbatra3@wisc.edu

Instructions: You can use this LaTeX file as a template to develop your homework. Submit your homework on time as a single pdf file to **Gradescope**. Although this is programming homework, you only need to hand in a pdf answer file. There is no need to submit the latex source or any code. You can choose any programming language (i.e., python, R, or MATLAB), as long as you implement the algorithm from scratch (e.g., do not use sklearn/Weka on questions 1 to 7 in section 2). Please check Piazza for updates about the homework.

1 A Simplified Decision Tree

You are to implement a decision-tree learner for classification. To simplify your work, this will not be a general purpose decision tree. Instead, your program can assume that

- each item has two continuous features $\mathbf{x} \in \mathbb{R}^2$
- the class label is binary and encoded as $y \in \{0, 1\}$
- data files are in plaintext with one labeled item per line, separated by whitespace:

$$\begin{array}{ccc} x_{11} & x_{12} & y_1 \\ & \dots & \\ x_{n1} & x_{n2} & y_n \end{array}$$

Your program should implement a decision tree learner according to the following guidelines:

- Candidate splits (j, c) for numeric features should use a threshold c in feature dimension j in the form of $x_j \geq c$.
- c should be on values of that dimension present in the training data; i.e. the threshold is on training points, not in between training points. You may enumerate all features, and for each feature, use all possible values for that dimension.
- You may skip those candidate splits with zero split information (i.e. the entropy of the split), and continue the enumeration.
- The left branch of such a split is the “then” branch, and the right branch is “else”.
- Splits should be chosen using information gain ratio. If there is a tie you may break it arbitrarily.
- The stopping criteria (for making a node into a leaf) are that
 - the node is empty, or
 - all splits have zero gain ratio (if the entropy of the split is non-zero), or
 - the entropy of any candidates split is zero
- To simplify, whenever there is no majority class in a leaf, let it predict $y = 1$.

2 Questions

1. (Our algorithm stops at pure labels) [10 pts] If a node is not empty but contains training items with the same label, why is it guaranteed to become a leaf? Explain. You may assume that the feature values of these items are not all the same.

Entropy of a set of class labels Y (where $|Y| = c$) is given by:

$$H(Y) = - \sum_{i=1}^c P(y_i) \log_2(P(y_i))$$

where $P(y_i)$ is the probability of class label y_i

Assume we have n training labels, all of which belong to the same label class y_0 . Let the threshold be t , then the splits X_{left} and X_{right} are as follows:

$$X_{left} = \{x \in \mathcal{X} | x_j < t\}$$

$$X_{right} = \{x \in \mathcal{X} | x_j \geq t\}$$

for both these splits, all items belong to the label class y_0 . Therefore for both splits,

- $p(y_0) = 1$
- $p(y_i) = 0; i \neq 0$

Therefore, the entropy of both the splits is as follows:

$$H(Y_{left}) = -(1 \cdot \log_2(1) + 0 \cdot \log_2(0)) = 0$$

$$H(Y_{right}) = -(1 \cdot \log_2(1) + 0 \cdot \log_2(0)) = 0$$

Hence, the candidate split entropy $H(Y|X) = 0$, as both $H(Y_{left})$ and $H(Y_{right})$ are 0.

This leads to the last stopping condition:

The node will become a leaf because the entropy of all candidates splits is zero.

Hence, it is guaranteed to become a leaf node.

2. (Our algorithm is greedy) [10 pts] Handcraft a small training set where both classes are present but the algorithm refuses to split; instead it makes the root a leaf and stop; Importantly, if we were to manually force a split, the algorithm will happily continue splitting the data set further and produce a deeper tree with zero training error. You should (1) plot your training set, (2) explain why. Hint: you don't need more than a handful of items.

The handcrafted training set that results in no further splits is as follows:

x_1	x_2	y
1	1	0
2	1	1

- (1) Plot of the training data

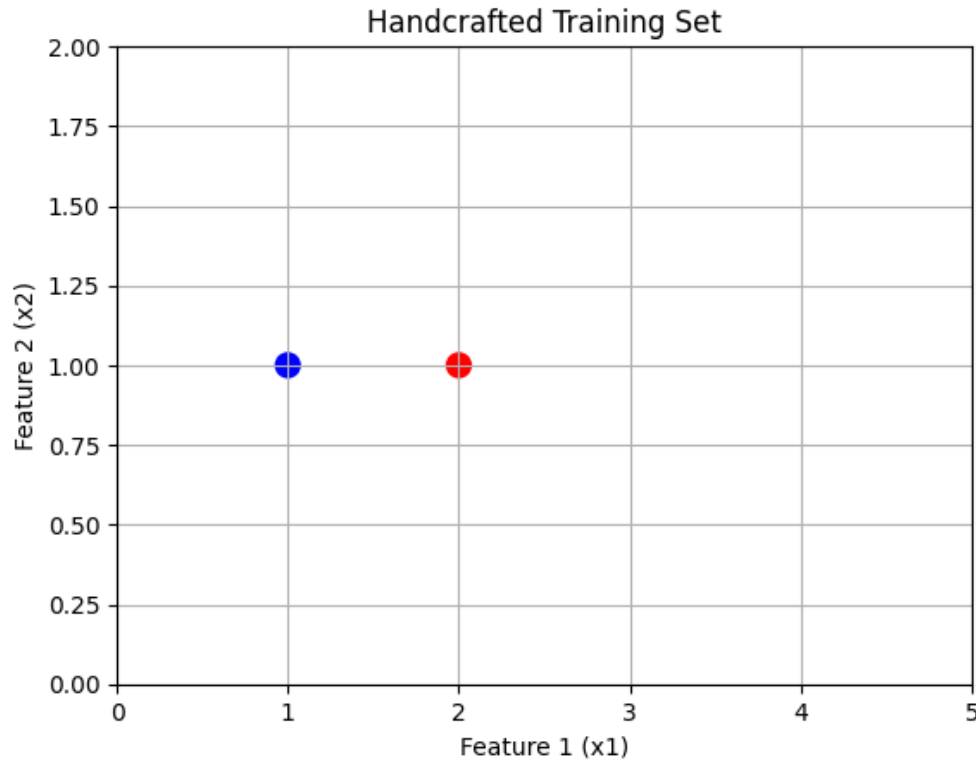


Figure 1: Handcrafted Dataset

(2) Explanation of why there will be no splitting.

Let us try to split on X_2 and X_1 using all possible values of thresholds.

Information Gain is defined as:

$$I(Y, X_j) = H(Y) - H(Y|X_j)$$

where $H(Y)$ is the entropy of class labels before the split and $H(Y|X_j)$ is the entropy of class labels given split on the feature X_j .

Also,

$$H(Y) = -(0.5 \log_2(0.5) + 0.5 \log_2(0.5)) = 1$$

- Now let's first split using feature X_2 . As all the elements are 1, the threshold is 1.

Hence,

$$H(Y|X_2) = -(0.5 \log_2(0.5) + 0.5 \log_2(0.5)) = 1 = H(Y)$$

Therefore, information gain $I(Y, X_2) = 1 - 1 = 0$. Hence, information gain ratio is 0. Therefore, according to the second stopping criteria, there will be no splitting.

- Now let's first split using feature X_1 . Let's try all different thresholds.

– threshold = 1, the split be as follows:

$$X_{left} = \{ \}; y = \{ \}$$

$$X_{right} = \{(1, 1)(2, 1)\}; y = \{0, 1\}$$

Therefore,

$$H(Y|X_1) = -(0.5 \log_2(0.5) + 0.5 \log_2(0.5)) = 1 = H(Y)$$

Therefore, information gain $I(Y, X_1) = 1 - 1 = 0$. Hence, information gain ratio is 0. Therefore, according to the second stopping criteria, there will be no splitting.

- threshold = 2, the split be as follows:

$$X_{left} = \{(1, 1)\}; y = \{0\}$$

$$X_{right} = \{(2, 1)\}; y = \{1\}$$

Therefore,

$$H(X_{left}) = H(X_{right}) = -(1 \cdot \log_2(1) + 0 \cdot \log_2(0)) = 0$$

Therefore, according to the last stopping condition, the entropy of all candidate splits is 0. Hence, there will be no splitting.

- threshold ≥ 3 , the split be as follows:

$$X_{left} = \{(1, 1)(2, 1)\}; y = \{0, 1\}$$

$$X_{right} = \{\}; y = \{\}$$

Therefore,

$$H(Y|X_1) = -(0.5 \log_2(0.5) + 0.5 \log_2(0.5)) = 1 = H(Y)$$

Therefore, information gain $I(Y, X_1) = 1 - 1 = 0$. Hence, information gain ratio is 0. Therefore, according to the second stopping criteria, there will be no splitting.

Hence, there can be no further splits on X_1 .

In-fact, following a similar line of proof we can show that any even number of points with diagonally opposite marked with opposite labels will result in no splitting.

For instance:

x_1	x_2	y
1	1	0
1	2	1
2	1	1
2	2	0

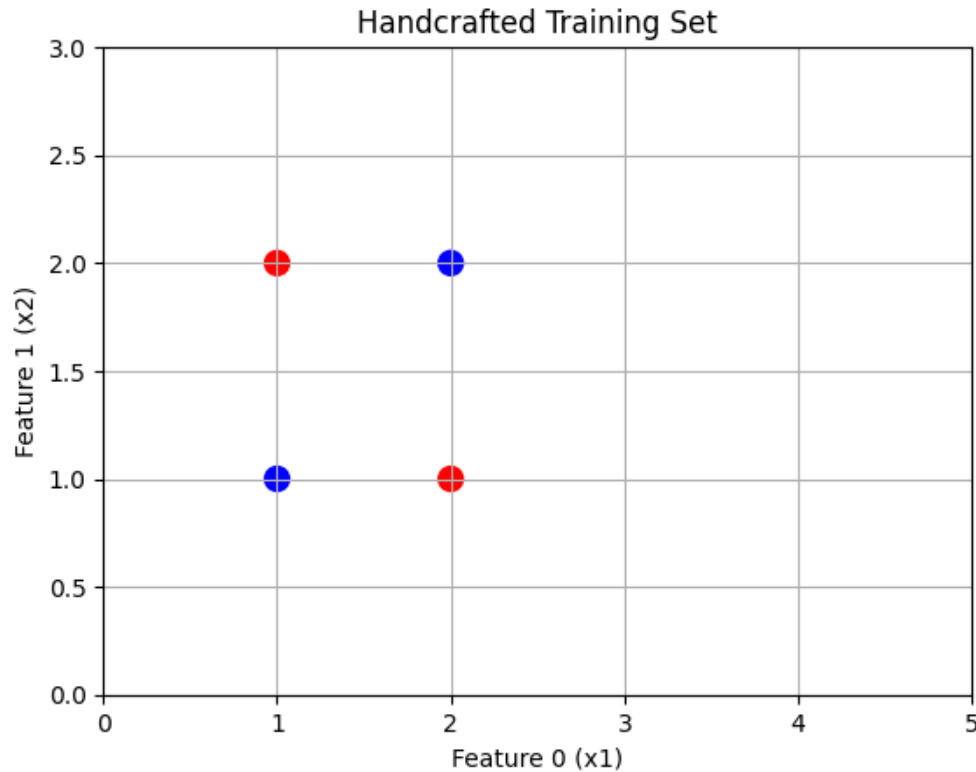


Figure 2: Handcrafted Dataset

These set of points will also satisfy the condition and no further splitting will occur.

3. (Information gain ratio exercise) [10 pts] Use the training set Druns.txt. For the root node, list all candidate cuts and their information gain ratio. If the entropy of the candidate split is zero, please list its mutual information (i.e. information gain). Hint: to get $\log_2(x)$ when your programming language may be using a different base, use $\log(x) / \log(2)$. Also, please follow the split rule in the first section.

Following the split rules, the table for thresholds, information gain ratio and information gains is as follows.

For feature x_0 :

Threshold	Information Gain Ratio	Information Gain
0.0	-	0
0.1	0.10051807676021828	0.04417739186726133

For feature x_1 :

Threshold	Information Gain Ratio	Information Gain
-2.0	-	0
-1.0	0.10051807676021828	0.04417739186726133
0.0	0.055953759631263526	0.03827452220629246
1.0	0.00578004220515232	0.004886164091842726
2.0	0.0011443495172767494	0.0010821659130775263
3.0	0.016411136842102134	0.016313165825732057
4.0	0.049749064181778546	0.04945207278939401
5.0	0.11124029586339801	0.10519553207004628
6.0	0.23609960614360798	0.19958702318968735
7.0	0.055953759631263526	0.03827452220629246
8.0	0.4301569161309807	0.18905266854301617

4. (The king of interpretability) [10 pts] Decision tree is not the most accurate classifier in general. However, it persists. This is largely due to its rumored interpretability: a data scientist can easily explain a tree to a non-data scientist. Build a tree from D3leaves.txt. Then manually convert your tree to a set of logic rules. Show the tree¹ and the rules.

Logical rules:

- $x_0 \geq 10.0$
 - True Predict: 1.0
 - False:
 - * $x_1 \geq 3.0$
 - True Predict: 1.0
 - False Predict: 0.0

Visual Representation of the tree:

¹When we say show the tree, we mean either the standard computer science tree view, or some crude plaintext representation of the tree – as long as you explain the format. When we say visualize the tree, we mean a plot in the 2D x space that shows how the tree will classify any points.

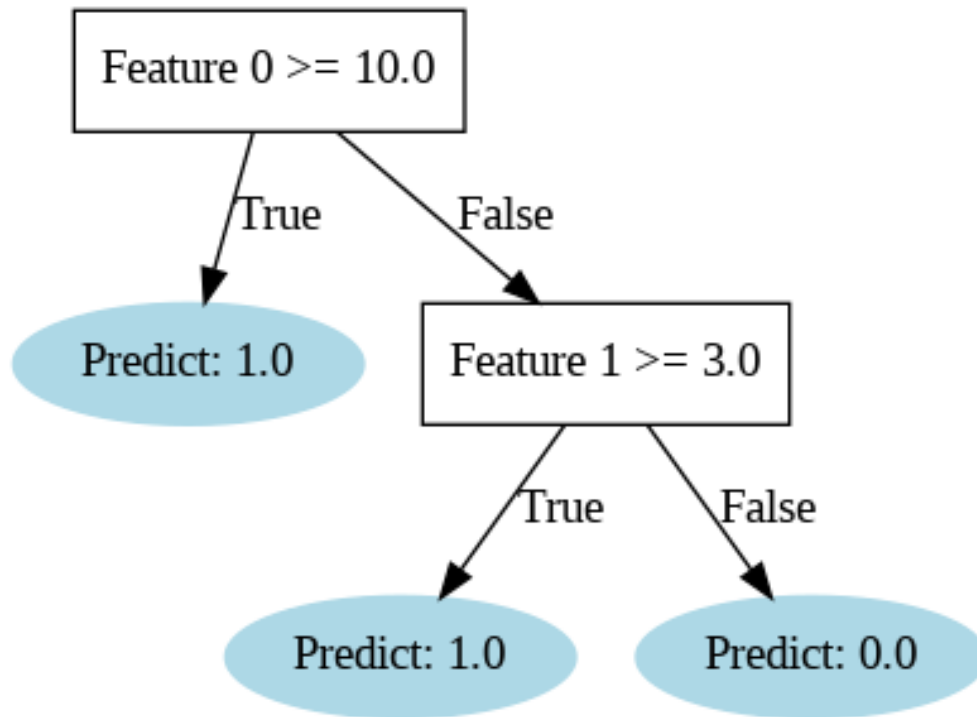


Figure 3: Decision Tree for D3leaves

5. (Or is it?) [10 pts] For this question only, make sure you DO NOT VISUALIZE the data sets or plot your tree's decision boundary in the 2D x space. If your code does that, turn it off before proceeding. This is because you want to see your own reaction when trying to interpret a tree. You will get points no matter what your interpretation is. And we will ask you to visualize them in the next question anyway.
- Build a decision tree on D1.txt. Show it to us in any format (e.g. could be a standard binary tree with nodes and arrows, and denote the rule at each leaf node; or as simple as plaintext output where each line represents a node with appropriate line number pointers to child nodes; whatever is convenient for you). Again, do not visualize the data set or the tree in the x input space. In real tasks you will not be able to visualize the whole high dimensional input space anyway, so we don't want you to "cheat" here.

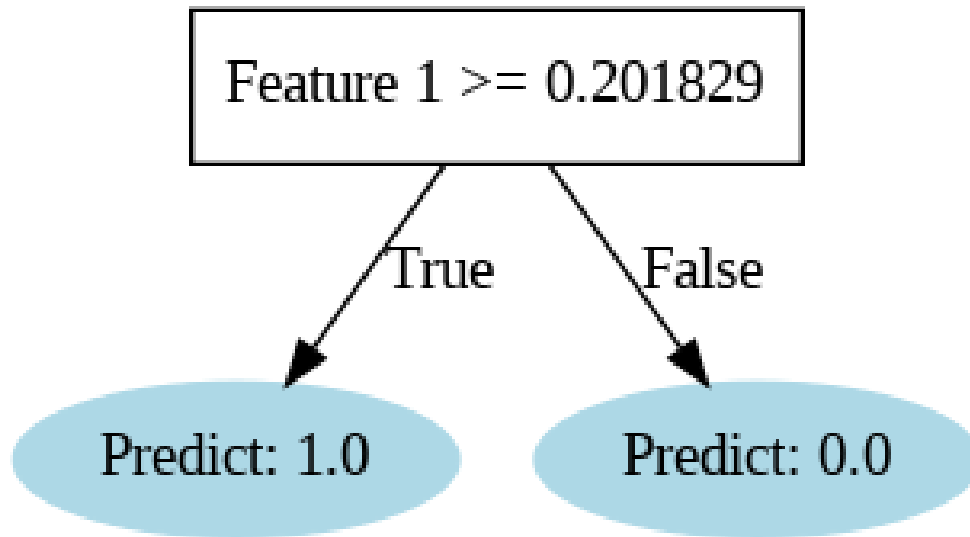


Figure 4: Decision Tree for D1.txt

- Look at your tree in the above format (remember, you should not visualize the 2D dataset or your tree's decision boundary) and try to interpret the decision boundary in human understandable English.

Looking at the simple decision tree, the decision boundary should be the straight line $x_1 = 0.201829$.

- Build a decision tree on D2.txt. Show it to us.

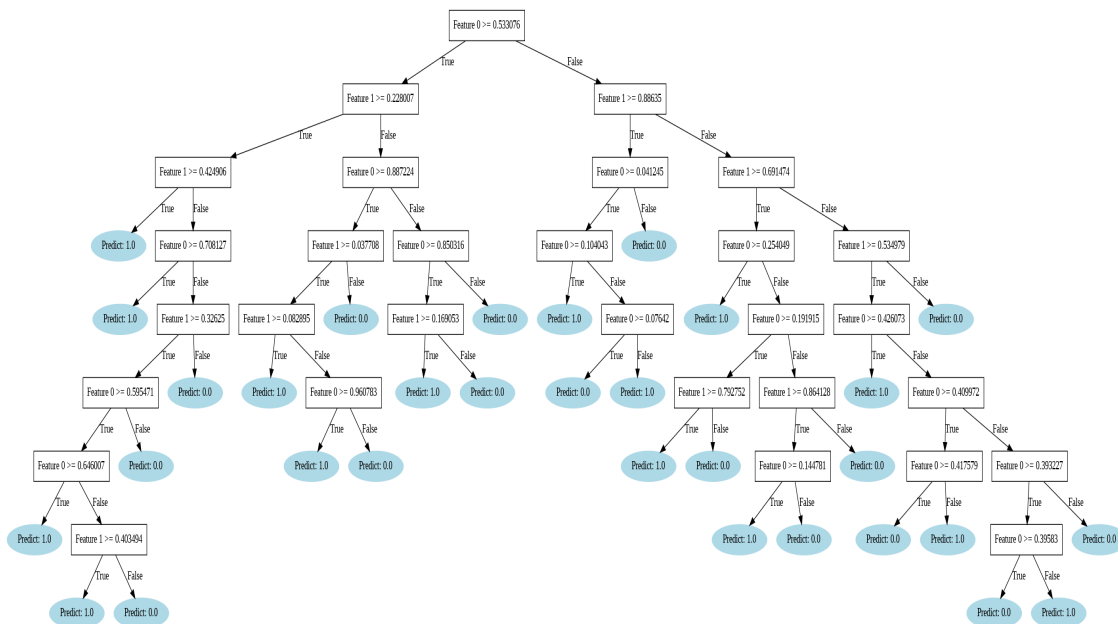


Figure 5: Decision Tree for D2.txt

- Try to interpret your D2 decision tree. Is it easy or possible to do so without visualization?

It is definitely more difficult to visualize the decision boundary for this decision tree without a scatter plot.

6. (Hypothesis space) [10 pts] For D1.txt and D2.txt, do the following separately:

- Produce a scatter plot of the data set.
- Visualize your decision tree's decision boundary (or decision region, or some other ways to clearly visualize how your decision tree will make decisions in the feature space).

Then discuss why the size of your decision trees on D1 and D2 differ. Relate this to the hypothesis space of our decision tree algorithm.

- D1.txt
 - Scatter Plot for D1.txt is given below

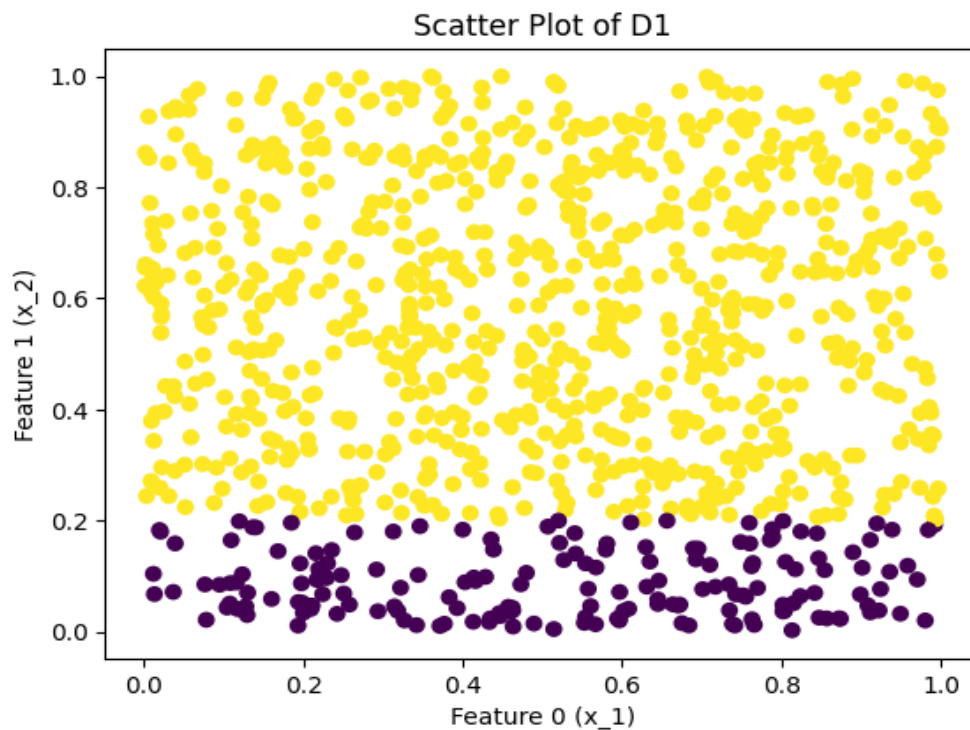


Figure 6: Scatter Plot for D1.txt

- Decision Boundary for D1.txt is given below

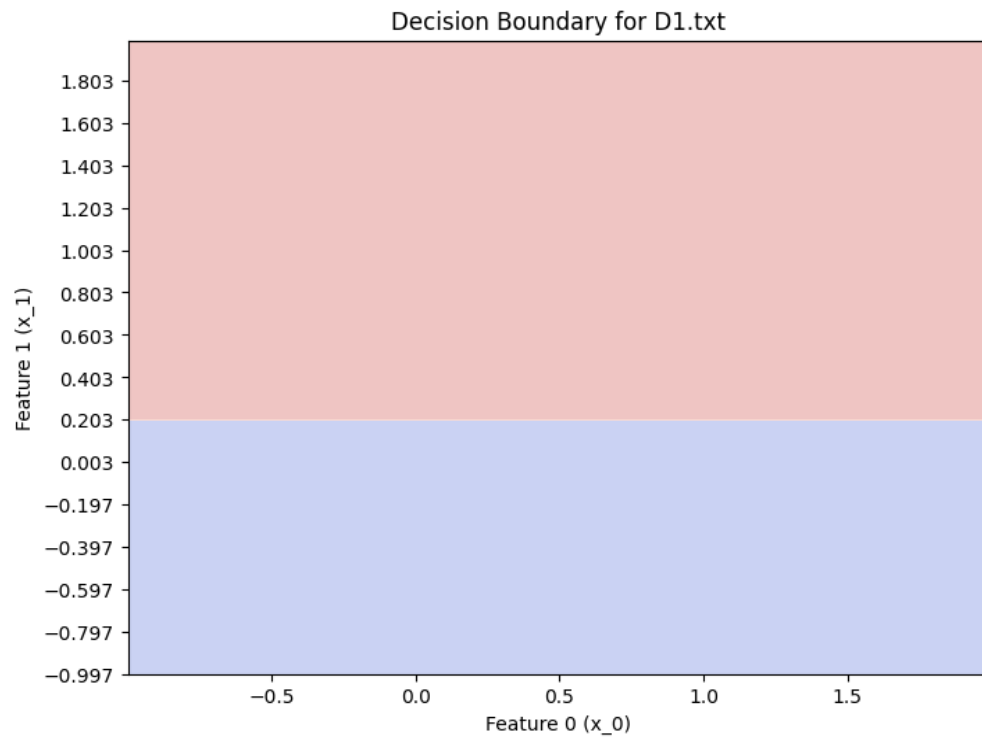


Figure 7: Decision Boundary for D1.txt

- D2.txt
 - Scatter Plot for D2.txt is given below

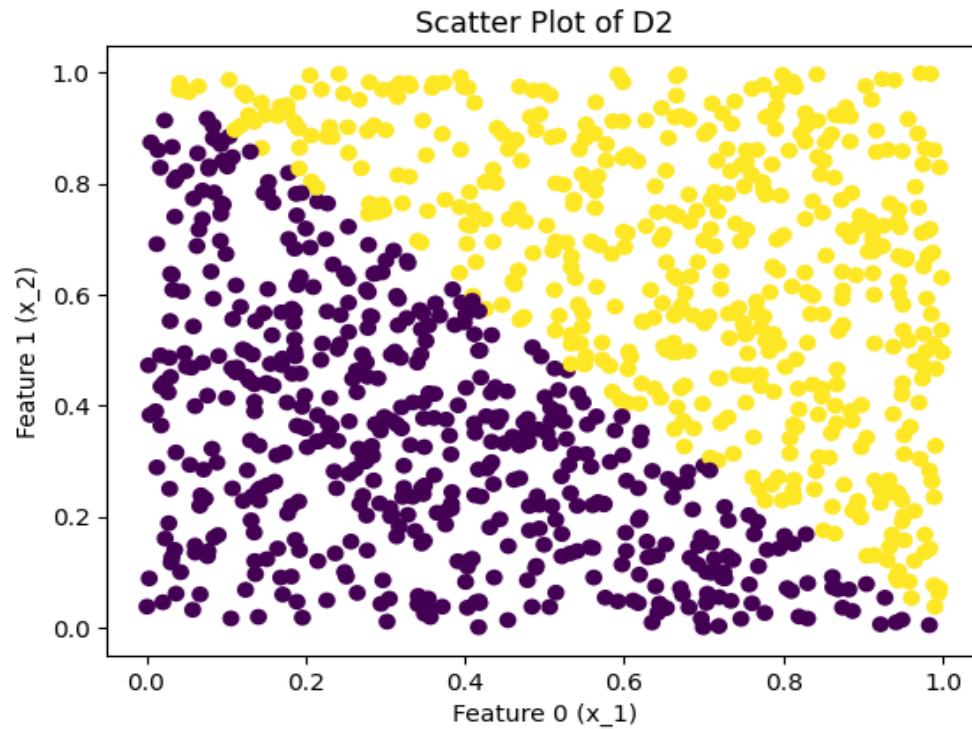


Figure 8: Scatter Plot for D2.txt

- Decision Boundary for D2.txt is given below

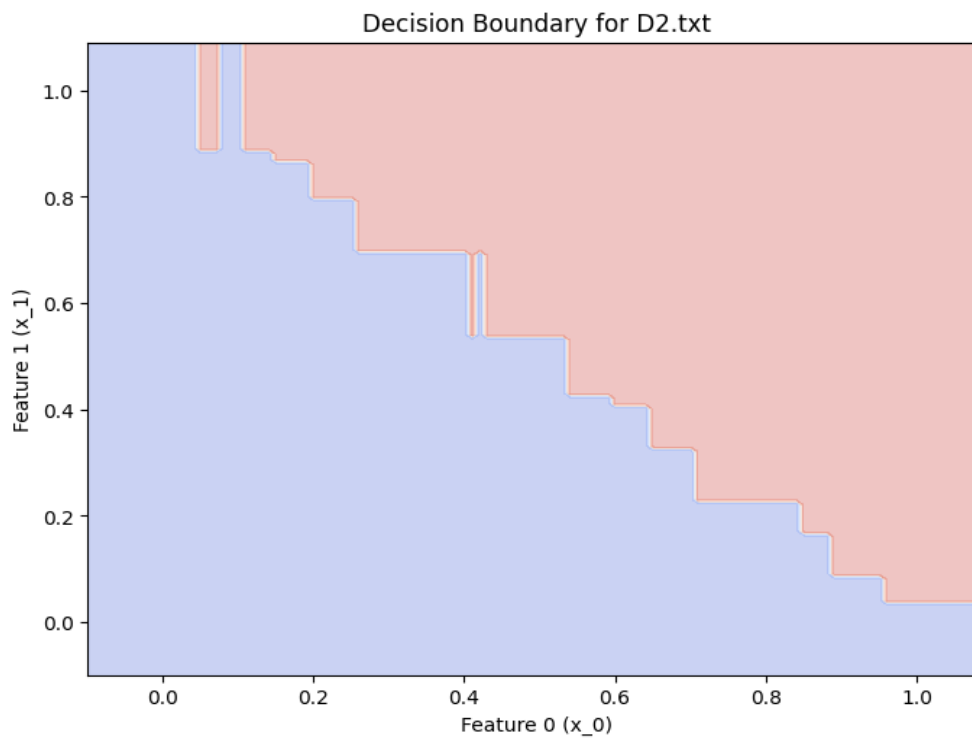


Figure 9: Decision Boundary for D2.txt

The sizes of the decision trees of D1.txt and D2.txt are different because it's easier to represent decision boundaries of the form $x_0 = c$ or $x_1 = c$ in a decision tree (where c is a constant). This kind of line translates to a single split in the decision tree representation.

However, the decision boundary of D2.txt, would be of the form $ax_0 + bx_1 = c$ where $a, b, c \neq 0$ and are constants. Approximating such a line in a decision tree would require many more splits which leads to the step size like decision boundary for D2.txt.

7. (Learning curve) [20 pts] We provide a data set Dbig.txt with 10000 labeled items. Caution: Dbig.txt is sorted.

- You will randomly split Dbig.txt into a candidate training set of 8192 items and a test set (the rest). Do this by generating a random permutation, and split at 8192.
- Generate a sequence of five nested training sets $D_{32} \subset D_{128} \subset D_{512} \subset D_{2048} \subset D_{8192}$ from the candidate training set. The subscript n in D_n denotes training set size. The easiest way is to take the first n items from the (same) permutation above. This sequence simulates the real world situation where you obtain more and more training data.
- For each D_n above, train a decision tree. Measure its test set error err_n . Show three things in your answer: (1) List n , number of nodes in that tree, err_n . (2) Plot n vs. err_n . This is known as a learning curve (a single plot). (3) Visualize your decision trees' decision boundary (five plots).

- List n , number of nodes and err_n .

n	number of nodes	err_n
32	9	0.1344
128	25	0.1294
512	59	0.0514
2048	115	0.0288
8192	277	0.0221

- Plot n vs. err_n

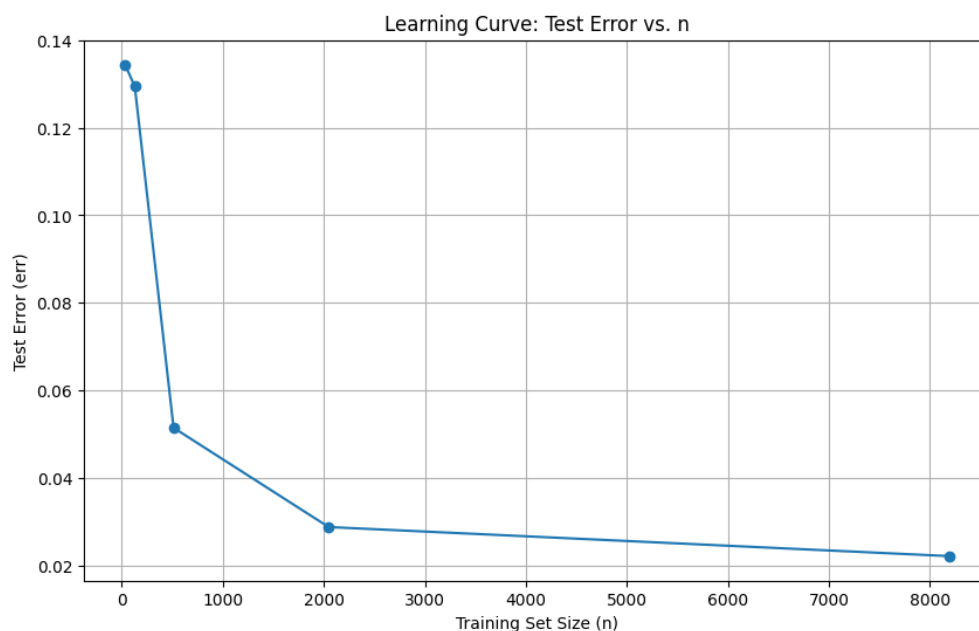


Figure 10: Plot for test error vs n

- Visualization of the decision trees' decision boundary (five plots).

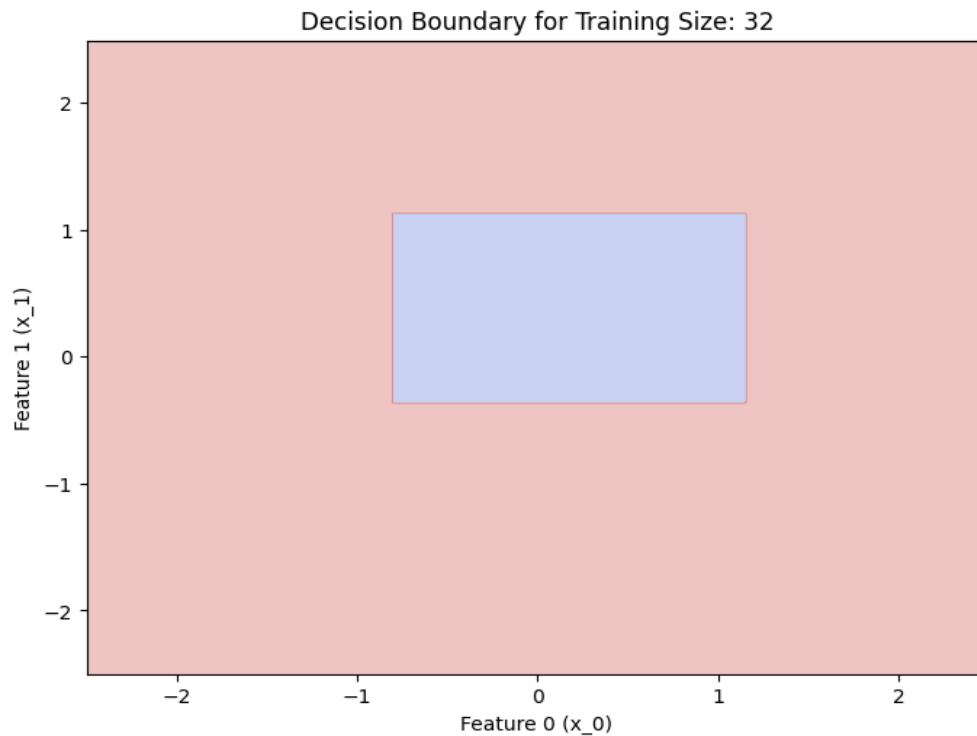


Figure 11: Decision Boundary for 32 training samples

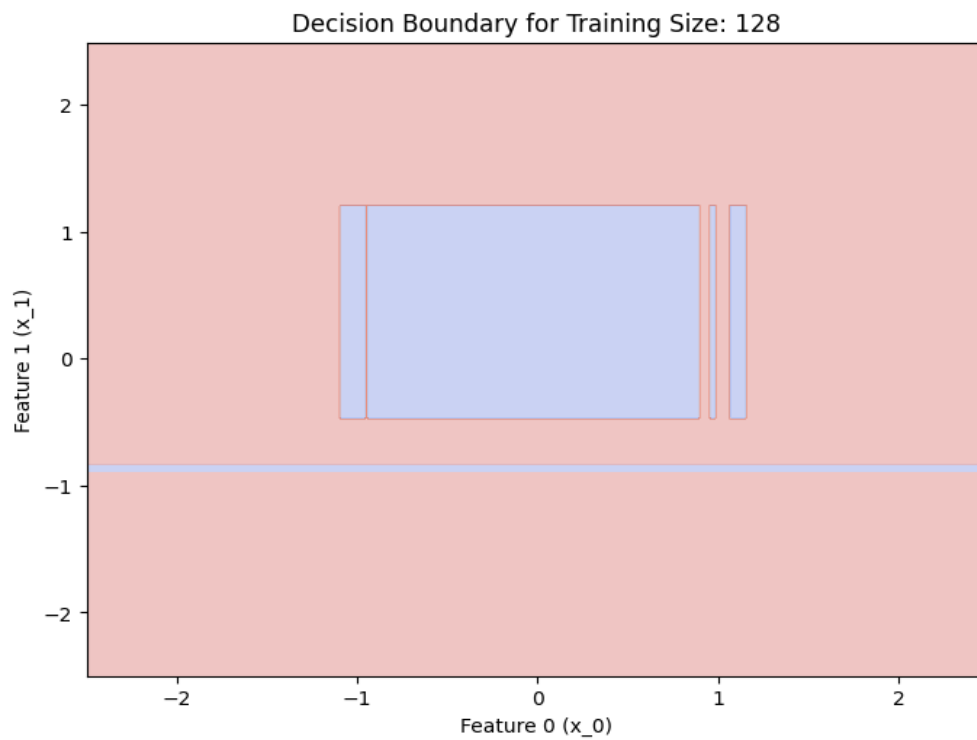


Figure 12: Decision Boundary for 128 training samples

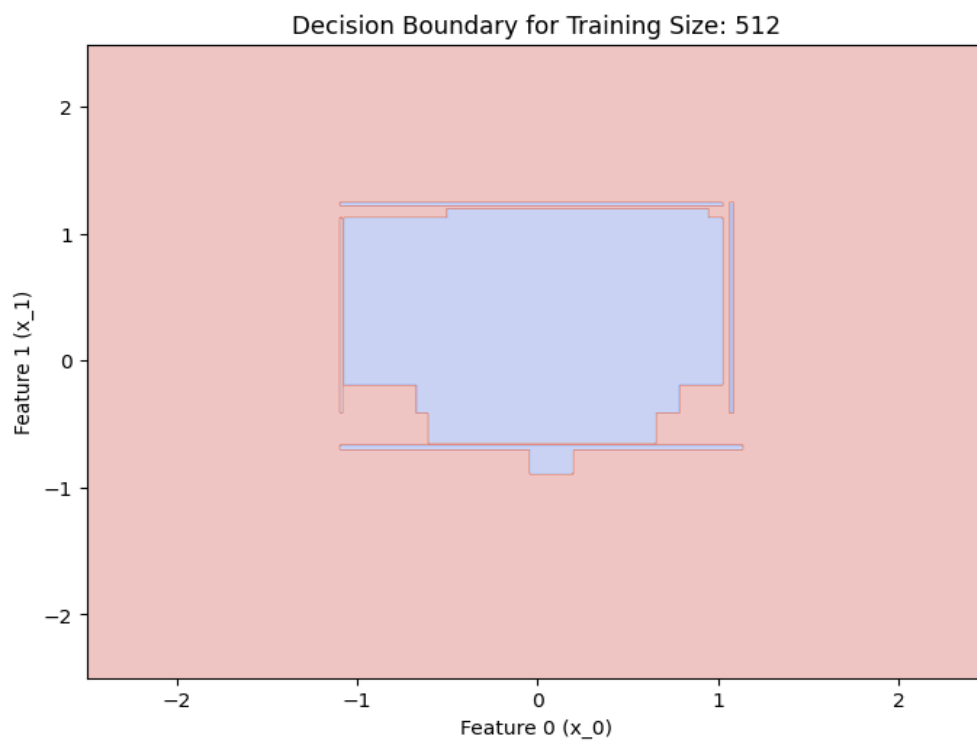


Figure 13: Decision Boundary for 512 training samples

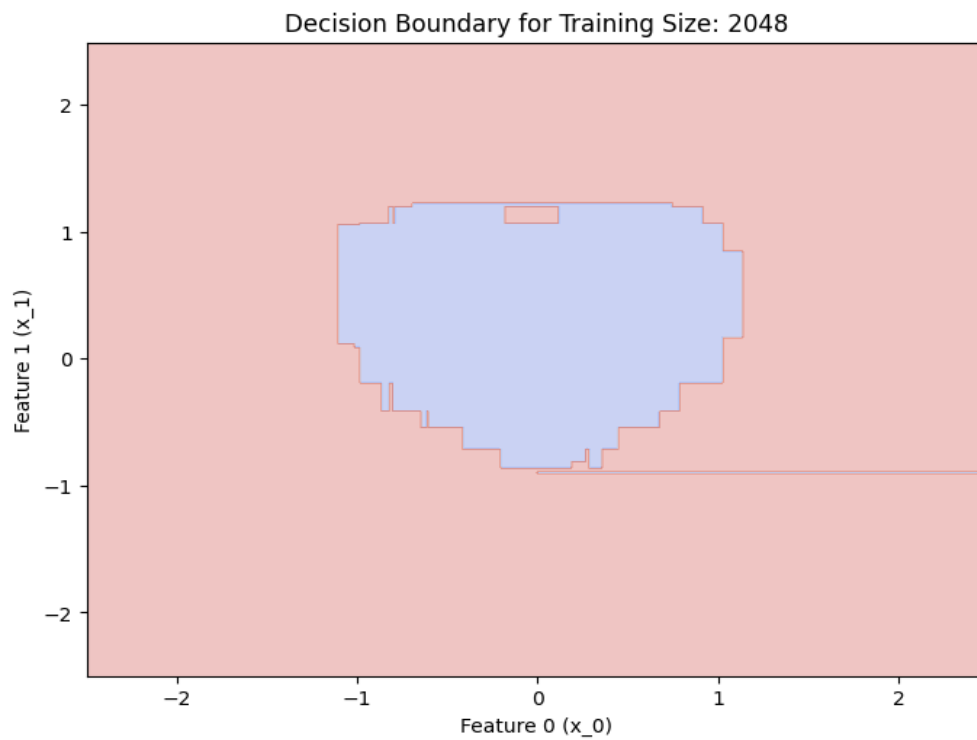


Figure 14: Decision Boundary for 2048 training samples

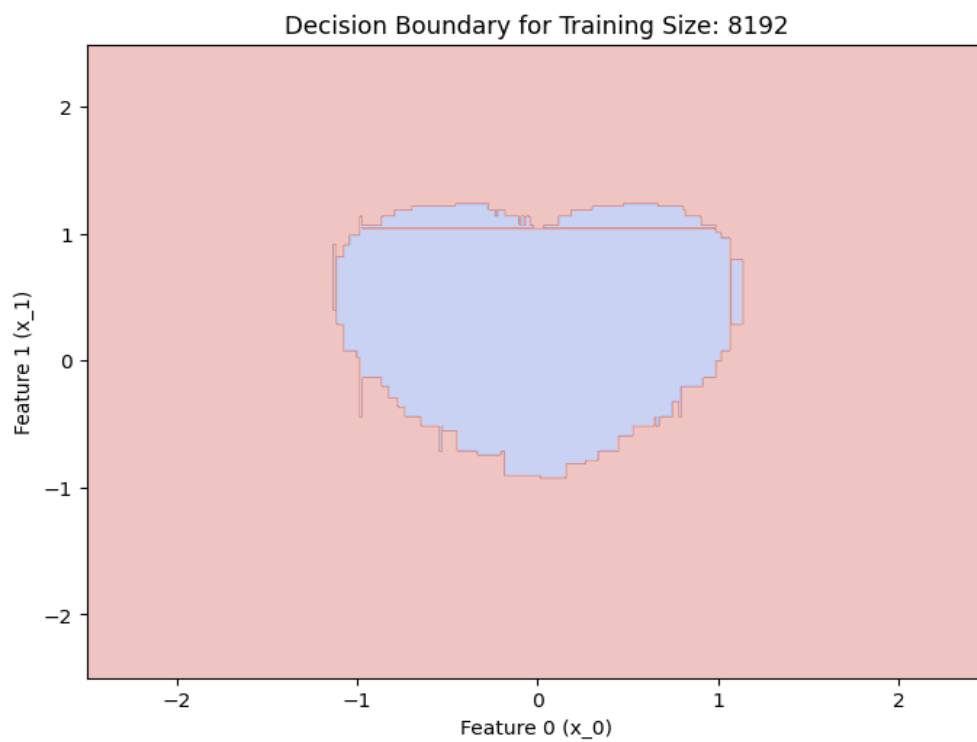


Figure 15: Decision Boundary for 8192 training samples

3 sklearn [10 pts]

Learn to use sklearn (<https://scikit-learn.org/stable/>). Use `sklearn.tree.DecisionTreeClassifier` to produce trees for datasets $D_{32}, D_{128}, D_{512}, D_{2048}, D_{8192}$. Show two things in your answer: (1) List n , number of nodes in that tree, err_n . (2) Plot n vs. err_n .

- List n , number of nodes and err_n .

n	number of nodes	err_n
32	9	0.1018
128	21	0.1073
512	59	0.0393
2048	105	0.0210
8192	229	0.0155

- Plot n vs. err_n

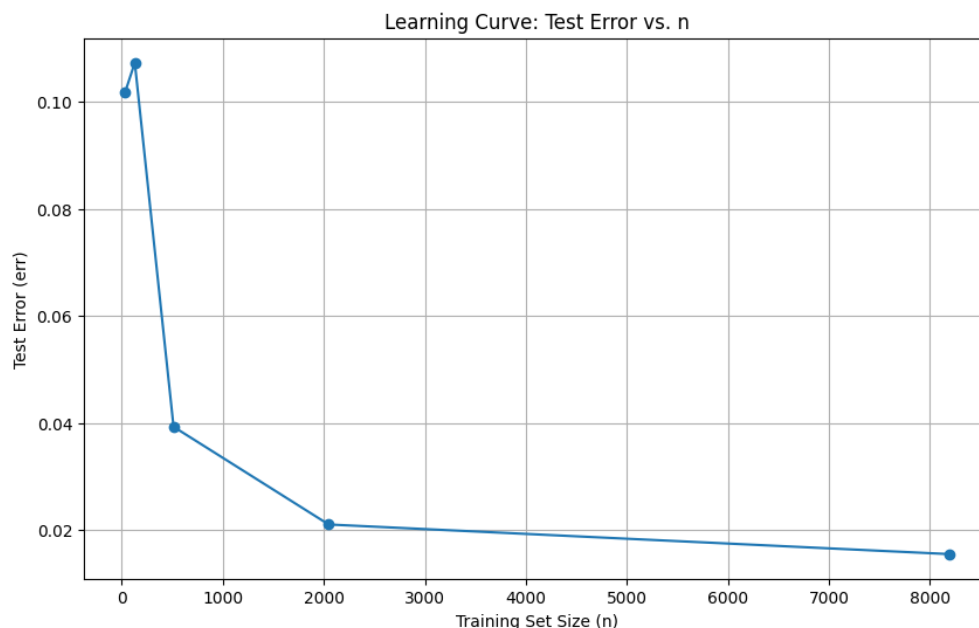


Figure 16: Plot for test error vs n using sklearn

4 Lagrange Interpolation [10 pts]

Fix some interval $[a, b]$ and sample $n = 100$ points x from this interval uniformly. Use these to build a training set consisting of n pairs (x, y) by setting function $y = \sin(x)$.

Build a model f by using Lagrange interpolation, check more details in https://en.wikipedia.org/wiki/Lagrange_polynomial and <https://docs.scipy.org/doc/scipy/reference/generated/scipy.interpolate.lagrange.html>.

Generate a test set using the same distribution as your test set. Compute and report the resulting model's train and test error. What do you observe? Repeat the experiment with zero-mean Gaussian noise ϵ added to x . Vary the standard deviation for ϵ and report your findings.

We try out with different errors:

Mean Squared Error

- Train Error: $2.7759275561729575 \times 10^{139}$
- Test Error: $2.7759275561729575 \times 10^{139}$

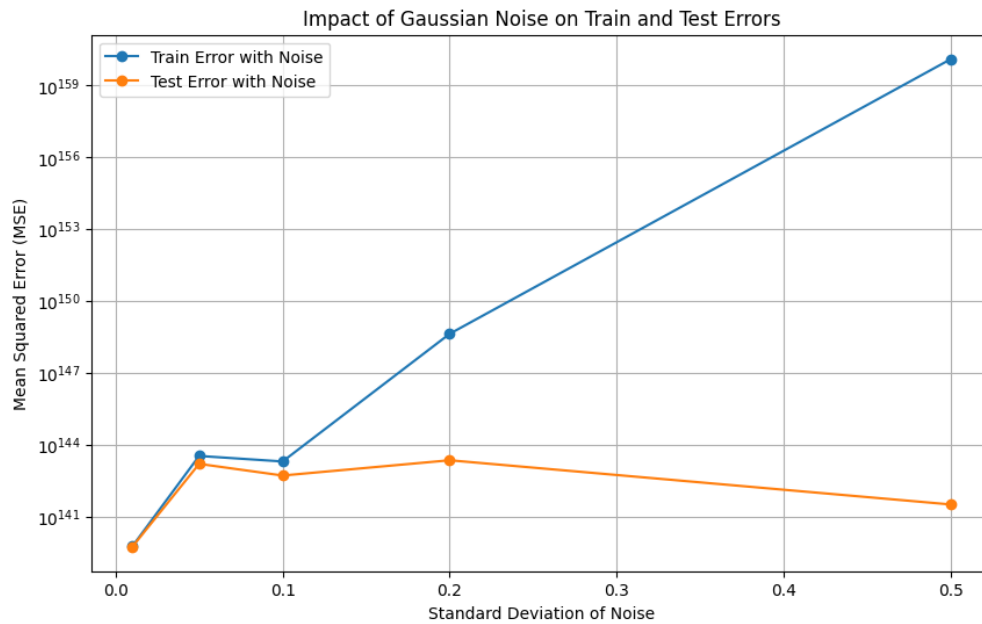


Figure 17: Lagrange Interpolation Model using MSE

Root Mean Squared Error

- Train Error: $5.268707200227545 \times 10^{69}$
- Test Error: $5.268707200227545 \times 10^{69}$

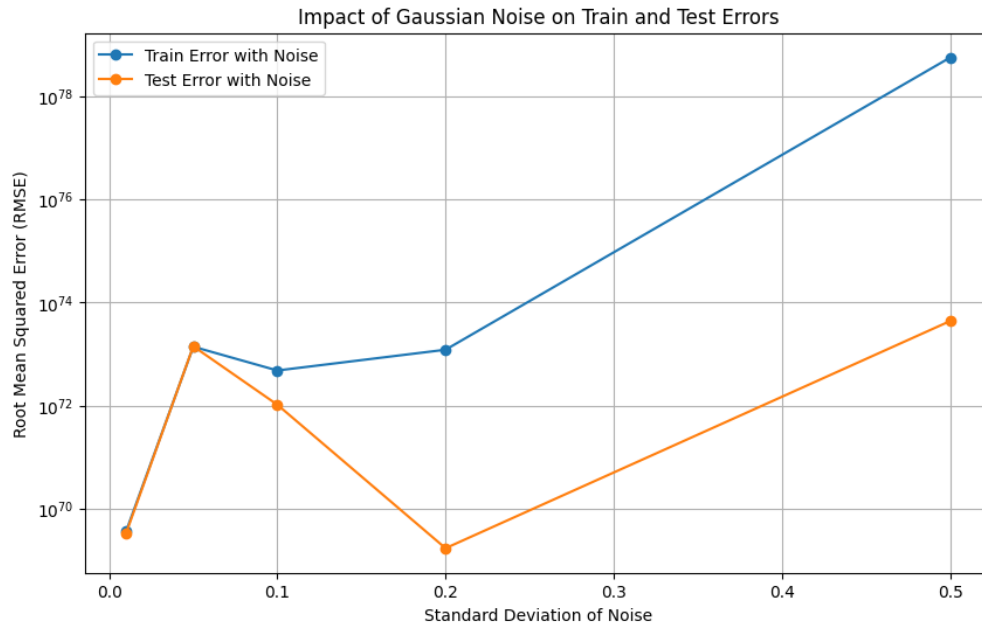


Figure 18: Lagrange Interpolation Model using RMSE

Mean Absolute Error

- Train Error: $8.968542165880735e+68$
- Test Error: $8.968542165880735e+68$

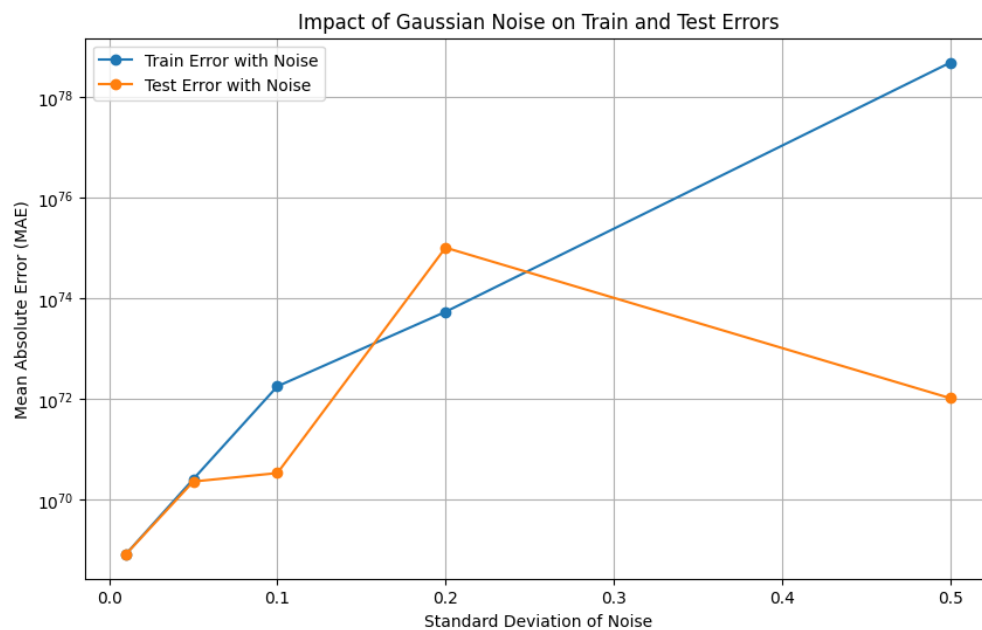


Figure 19: Lagrange Interpolation Model using MAE

Findings

- Lagrange interpolation tends to be highly sensitive to the number of points. With too many points, the interpolating polynomial can oscillate wildly between the points (known as Runge's phenomenon), leading to poor generalization, thus high train and test losses.
- As Gaussian noise is added to the input values both the train and test errors generally tend to increase.
- Even with small amounts of noise (low standard deviations), the test error increases significantly, which shows that the model is not robust to noise.